MINISTRY OF EDUCATION AND TRAINING

FPT UNIVERSITY

Capstone Project Document

Chart QA: +

Chart QA	
Group Members	noname
	noname
	noname
Supervisor	Huynh Van Thong Not applicable for this Course
Capstone Project code	AIP491-SP26

HCMC, April 2026

Contents

List of Figures iii

List of Tables iv

1 Introduction 1
1.1 Background . . . 1
1.2 Objective . . . 1
1.3 Problem Statement . . . 2
1.4 Scope and Limitations . . . 2

2 Project Management Plan 4
2.1 Team Structure and Roles . . . 4
2.2 Communication Plan . . . 4

3 Literature Review 5
3.1 End-to-End Multimodal Approaches . . . 5
3.2 Classical Geometric Approaches . . . 5
3.3 Object Detection for Document Analysis . . . 5
3.4 OCR and Text Extraction . . . 6
3.5 Small Language Models and Parameter-Efficient Fine-Tuning . . . 6
3.6 Summary and Research Gap . . . 6

4 Methodology 8
4.1 Data . . . 8
4.1.1 Data Collection Pipeline . . . 8
4.1.2 SLM Training Dataset . . . 9
4.1.3 SLM Fine-Tuning Configuration . . . 9
4.2 Approach . . . 11
4.2.1 Hybrid Neuro-Symbolic Design . . . 11
4.2.2 5-Stage Pipeline Architecture . . . 11
4.2.3 AI Routing with Fallback Chains . . . 12
4.2.4 Configuration-Driven Design . . . 12
4.3 Vietnamese Language Integration Strategy . . . 12
4.3.1 Localization Architecture . . . 12
4.3.2 OCR for Vietnamese Diacritical Marks . . . 13
4.3.3 Vietnamese Keyword Mapping for Role Classification . . . 13
4.3.4 Vietnamese Prompt Templates and QA Generation . . . 14
4.3.5 SLM Base Model Selection for Vietnamese . . . 14
4.3.6 Deployment Plan for Vietnamese Integration . . . 14

5 System Design and Implementation 15
5.1 Layered Architecture . . . 15
5.2 Stage 1: Ingestion and Sanitation . . . 15
5.3 Stage 2: Detection and Localization . . . 16
5.4 Stage 3: Structural Analysis (Extraction) . . . 16
5.4.1 Preprocessor . . . 16

5.4.2 Skeletonizer . . . 17
5.4.3 Vectorizer . . . 17
5.4.4 OCR Engine . . . 17
5.4.5 Element Detector . . . 18
5.4.6 Geometric Mapper . . . 18
5.4.7 Classifier . . . 18
5.5 Stage 4: Semantic Reasoning . . . 18
5.6 Stage 5: Insight and Reporting . . . 19
5.7 AI Routing Layer . . . 19
5.8 Error Handling and Extensibility . . . 20
5.9 Vietnamese Language Implementation . . . 20
5.9.1 Language-Agnostic vs. Language-Dependent Components . . . 20
5.9.2 OCR Engine: Vietnamese Configuration Path . . . 22
5.9.3 Stage 3: Bilingual Role Classification . . . 22
5.9.4 Stage 4: Vietnamese Prompt Architecture . . . 22
5.9.5 Data Factory: Vietnamese QA Generation . . . 23

6 Results and Discussion **24**
6.1 Results and Analysis . . . 24
6.1.1 Stage 2: Chart Detection (YOLOv8m) . . . 24
6.1.2 Stage 2b: Chart Type Classification (Ablation Study) . . . 24
6.1.3 Stage 3: Structural Extraction . . . 25
6.1.4 End-to-End Pipeline Performance . . . 26
6.1.5 Stage 4: SLM Hyperparameter Ablation Study . . . 26
6.1.6 Code Quality . . . 28
6.2 Discussion . . . 29
6.2.1 Hybrid Architecture Validates Core Hypothesis . . . 29
6.2.2 Comparison with Related Work . . . 29
6.2.3 Modularity Enables Iterative Improvement . . . 30
6.2.4 Core-First Strategy Validates Vietnamese Readiness . . . 30
6.2.5 Limitations of Current Results . . . 30
6.3 Recommendations . . . 31

7 Conclusion **32**
7.1 Summary of Contributions . . . 32
7.2 Achievement of Objectives . . . 32
7.3 Future Work . . . 33

References **34**

List of Figures

1 Data collection pipeline: from arXiv papers to SLM training dataset. Each stage filters and transforms the data, reducing volume while increasing quality. 8
2 Data pipeline funnel: volume at each stage from raw PDFs to final dataset. . . . 10
3 Comparison of traditional end-to-end approach (top) vs. Geo-SLM hybrid neuro-symbolic approach (bottom). Geo-SLM uses geometric measurement for precision and SLM reasoning for semantic understanding. 11
4 Geo-SLM 5-stage pipeline architecture with input/output types and representative technologies at each stage. 11
5 Geo-SLM 5-layer system architecture. Each layer depends only on layers below it. The Core Engine is independent of the serving layer. 15
6 Stage 3 extraction pipeline: 7 submodules process the cropped chart image sequentially. Each submodule has an independent configuration and can be toggled on/off. 17
7 AI Router with Adapter Pattern and per-task fallback chains. Each provider implements the common `BaseAIAdapter` interface. 19
8 Feature extraction quality by chart type: axis information coverage (left axis) and mean OCR confidence (right axis). 26
9 Estimated processing time per chart by pipeline stage. OCR in Stage 3 accounts for approximately 65% of total time. 26

List of Tables

1 Comparison of existing chart understanding methods. 6
2 Data pipeline scale at each processing stage. 8
3 Chart type distribution in the classified dataset (32,364 charts). 9
4 SLM training dataset v3 split statistics. 9
5 SLM fine-tuning configuration (QLoRA). 10
6 Pipeline stages: input, output, and key technologies. 12
7 Vietnamese localization touchpoints across the pipeline. 13
8 Technology stack overview. 15
9 AI provider configuration. 20
10 Language dependency analysis of pipeline components. 21
11 YOLOv8m chart detection results. 24
12 Chart classifier ablation: backbone, resolution, and class scope. 24
13 EfficientNet-B0 3-class per-class evaluation (test set, $n = 1{,}952$). 25
14 Stage 3 feature quality by chart type. 25
15 Token length distribution of training conversations (228,494 samples, Llama-3.2-1B tokenizer). Mean = 224, median = 223, P99 = 412, max = 1,576. 27
16 Ablation study configurations. Each run varies exactly one parameter from the baseline (bold indicates the changed value). 27
17 Ablation study results (Llama-3.2-1B, 800 samples, 10 epochs). Bold indicates best value per column. 27
18 Overfitting analysis: train–eval loss gap by configuration. 28
19 Test suite results by component. 28
20 Comparison with related chart understanding methods. 29
21 Achievement of project objectives. 32

1 Introduction

1.1 Background

Charts and graphs are among the most common methods for presenting quantitative information in scientific publications, business reports, and data dashboards. A typical research paper contains between two and ten chart images, each encoding numerical relationships that are critical for understanding experimental results. Despite advances in machine learning and natural language processing, the task of automatically extracting structured numerical data from chart images remains a significant challenge.

Current approaches to automatic chart understanding fall into two categories. The first category uses **end-to-end multimodal models** such as DePlot [1] and MatCha [2], which feed chart images directly into large vision-language models that predict data tables or answer questions. While these approaches benefit from the generalization capabilities of large models, they suffer from a fundamental limitation: the model *estimates* numerical values from pixel patterns rather than *measuring* them from geometric structures. This leads to hallucination errors where the model fabricates plausible but incorrect data points.

The second category relies on **rule-based geometric analysis**, such as ReVision [3], which uses Hough transforms and template matching to extract values from specific chart types. These methods are precise when applicable but lack generalizability across chart types and fail on noisy or complex layouts.

Neither paradigm alone is sufficient. End-to-end models lack geometric precision, while rule-based systems lack the perceptual flexibility of neural networks and the semantic understanding of language models. This gap motivates a **hybrid neuro-symbolic approach** that combines the strengths of both paradigms.

Furthermore, existing chart understanding systems are designed almost exclusively for English-language charts. In the Vietnamese context, academic publications, government reports, and business dashboards increasingly use charts with Vietnamese labels, axis titles, and legends – including diacritical marks (e.g., "Doanh thu", "Tỷ lệ phần trăm", "Biểu đồ cột"). No published system addresses Vietnamese chart understanding with structured data extraction and Vietnamese-language question answering. This project targets this gap by designing the pipeline as a **language-extensible** system: the core geometric and neural components are language-agnostic, while OCR configuration, prompt templates, and SLM fine-tuning data serve as localization layers that can be switched to Vietnamese without modifying core logic.

1.2 Objective

This project aims to design and implement **Geo-SLM**, a hybrid AI system for extracting structured numerical data from chart images. The system combines three complementary paradigms:

1. **Deep Learning for perception**: YOLOv8 [4] for chart detection and ResNet-18 [5] for chart type classification.

2. **Classical geometry for precision**: RANSAC-based axis calibration [6], Ramer-Douglas-Peucker polyline simplification [7, 8], and skeleton-based structural analysis [9] using OpenCV [10].

3. **Small Language Model (SLM) for reasoning**: A fine-tuned Qwen-2.5-1.5B [11] using QLoRA [12] for OCR correction, value validation, and natural language description generation.

The specific objectives are:

- Achieve chart detection mAP@0.5 > 85% and classification accuracy > 90%.
- Extract structured data (axis values, data series, labels) from 8 chart types with extraction confidence > 90%.
- Build a large-scale training dataset (> 200,000 samples) from academic papers for SLM fine-tuning.
- Demonstrate that the hybrid geometric-plus-AI approach outperforms pure end-to-end methods in numerical accuracy while running locally on consumer hardware (NVIDIA RTX 3060, 6 GB VRAM).
- Enable **Vietnamese-language chart question answering**: the system must process charts containing Vietnamese text and support interactive Q&A in Vietnamese, leveraging the extensible pipeline architecture.

1.3 Problem Statement

Given a document (PDF or image) containing one or more chart images, the system must:

- **Input**: A PDF document or image file (PNG, JPG) containing charts.
- **Output**: For each detected chart, a structured JSON object containing: chart type, title, axis labels, data series (named lists of (x, y) points), natural language description, and confidence scores with full traceability to source coordinates. The system must also support Vietnamese-language question answering over extracted chart data.
- **Chart types**: 8 categories – line, bar, scatter, pie, histogram, box, heatmap, and area.
- **Performance targets**: Detection mAP@0.5 > 85%; Classification accuracy > 90%; End-to-end extraction confidence > 80%.
- **Deployment constraint**: Must run on a single consumer GPU (RTX 3060, 6 GB VRAM) without requiring cloud API access for core inference.

1.4 Scope and Limitations

In scope:

- Charts from academic papers (arXiv cs.CV, cs.LG, cs.AI) and Vietnamese-language documents.
- 8 chart types: line, bar, scatter, pie, histogram, box, heatmap, area.
- PDF and raster image inputs (PNG, JPG, TIFF, BMP).
- Structured data extraction with JSON output.
- Local inference using fine-tuned SLM (1.5B parameters).
- Vietnamese-language OCR, chart Q&A, and description generation.

Out of scope:

- Business dashboards, presentation slides, and infographics.
- 3D charts, Sankey diagrams, and multi-panel composite figures.
- Real-time streaming or interactive chart manipulation.
- DOCX and HTML input formats (planned for future work).

Known limitations:

- The core training data is sourced from English academic publications; Vietnamese chart data is generated through the localization pipeline (see Section 4.3).
- Axis calibration requires at least two readable numeric tick labels; charts without visible axis numbers produce no calibrated values.
- The SLM fine-tuning is ongoing at the time of writing; final SLM evaluation results (both English and Vietnamese) will be supplemented upon completion.
- Vietnamese OCR accuracy depends on diacritical mark recognition quality, which varies across OCR engines and font styles.

2 Project Management Plan

This project is developed as an individual capstone by a single student.

2.1 Team Structure and Roles

The project is completed by one member who handles all responsibilities: system architecture design, data collection, model training, pipeline implementation, testing, documentation, and thesis writing. Version control is managed through Git with conventional commit messages, and progress is tracked via weekly reports to the course coordinator.

2.2 Communication Plan

Weekly progress updates are submitted following the course schedule. The project documentation (architecture documents, progress reports, and experiment logs) is maintained in the repository under `docs/`. Collaboration with the thesis advisor uses Overleaf for LaTeX document review.

3 Literature Review

This section reviews existing methods for automatic chart understanding, organized by approach paradigm. We analyze the strengths and limitations of each category to motivate the hybrid design of Geo-SLM.

3.1 End-to-End Multimodal Approaches

Recent advances in vision-language models have enabled end-to-end chart understanding systems that process chart images directly without explicit geometric decomposition.

DePlot [1] employs a two-stage approach: a Pix2Struct model translates the chart image into a linearized data table, which is then processed by a large language model (LLM) for question answering. DePlot achieves competitive results on the ChartQA benchmark [13] with one-shot prompting. However, because the Pix2Struct model estimates tabular values from visual patterns, it is prone to hallucination – producing plausible but numerically incorrect entries, especially for charts with dense data points or overlapping series.

MatCha [2] extends the Pix2Struct architecture with two pretraining objectives: math reasoning and chart derendering. The derendering task trains the model to generate the underlying data table from a chart image, improving numerical grounding. MatCha achieves strong results on both ChartQA and PlotQA [14] benchmarks. Despite these improvements, the model remains a large (282M+ parameters) encoder-decoder that requires significant compute resources and does not provide interpretable intermediate outputs.

ChartReader [15] proposes a unified framework that detects chart components (bars, lines, legend entries) using Faster R-CNN, then applies heuristic rules to reconstruct the data table. This approach provides more interpretable outputs than DePlot or MatCha, but its component detection relies heavily on annotated training data and struggles with unconventional chart layouts.

3.2 Classical Geometric Approaches

Before deep learning, chart understanding relied on classical image processing and geometric analysis.

ReVision [3] classifies charts using visual features and extracts data from bar charts using Hough line detection. While fully deterministic and interpretable, ReVision supports only a single chart type (bar) and requires strict assumptions about chart layout (aligned bars, visible grid lines). The method fails on scatter plots, line charts, and charts with non-standard aesthetics.

Other traditional approaches use contour detection, template matching, and projection profiles to identify chart elements. These methods offer high precision for well-formatted charts but lack robustness against the visual diversity found in real-world academic publications.

3.3 Object Detection for Document Analysis

The YOLO family [16, 4] has been widely adopted for object detection tasks in document analysis. YOLOv8 provides real-time detection with competitive accuracy, making it suitable for chart region localization within full-page document images. For chart detection specifically, the single-class detection strategy (detecting "chart" as one class) with a separate classification stage has been shown to outperform multi-class detection, since YOLO excels at localization while dedicated classifiers (e.g., ResNet-18 [5]) provide higher type-classification accuracy.

3.4 OCR and Text Extraction

Optical character recognition is fundamental to extracting textual information from charts, including axis labels, titles, legend entries, and tick mark values. PaddleOCR [17] and EasyOCR [18] are two widely-used open-source frameworks supporting multi-language text detection and recognition. In the context of chart analysis, OCR faces unique challenges: text may be rotated (Y-axis labels), overlapping (dense tick marks), or embedded as vector graphics (invisible to pixel-based OCR). Post-processing heuristics – such as spatial role classification (assigning text to title, axis, legend, or value roles based on position) and common-error correction (e.g., "O" to "0", "l" to "1") – are essential for reliable chart text extraction.

3.5 Small Language Models and Parameter-Efficient Fine-Tuning

Large language models have demonstrated strong reasoning capabilities for chart understanding, but their deployment on consumer hardware is impractical. Small language models (SLMs) with 1–3 billion parameters offer a practical alternative. Qwen-2.5 [11] provides instruction-tuned models starting at 0.5B parameters, with the 1.5B variant balancing capability and efficiency.

Parameter-efficient fine-tuning methods allow adapting these models to domain-specific tasks without full retraining. LoRA [19] introduces low-rank decomposition of weight update matrices, training only 0.1–1% of model parameters. QLoRA [12] extends LoRA with 4-bit quantization (NF4 format), reducing memory requirements to as low as 4 GB VRAM for a 1.5B-parameter model. This enables fine-tuning and inference on consumer GPUs such as the NVIDIA RTX 3060 (6 GB VRAM).

3.6 Summary and Research Gap

Table 1 summarizes the comparison of existing approaches.

Table 1: Comparison of existing chart understanding methods.

Method	Approach	Value Acc.	Types	Model Size	Explainable
DePlot [1]	Pix2Struct + LLM	∼85%	5+	282M+	No
MatCha [2]	Chart pre-training	∼80%	5+	282M+	No
ChartReader [15]	Detection + rules	∼75%	3	N/A	Partial
ReVision [3]	Hough transform	∼70%	1	N/A	Yes
Geo-SLM (Ours)	**Hybrid neuro-symbolic**	**Target >95%**	**8**	**1.5B**	**Yes**

The key research gap is the absence of a system that combines:

1. **Neural perception** for robust detection and classification across chart types,
2. **Geometric measurement** for deterministic, traceable value extraction, and
3. **Local SLM reasoning** for semantic understanding without cloud dependency.

Geo-SLM addresses this gap by integrating all three paradigms in a modular, 5-stage pipeline where each component operates on well-defined schema contracts, enabling independent development, testing, and replacement of any component.

4 Methodology

This section describes the data collection pipeline and the overall approach used in Geo-SLM. The data subsection covers the construction of the training dataset from raw academic papers. The approach subsection presents the 5-stage hybrid pipeline architecture.

4.1 Data

4.1.1 Data Collection Pipeline

The training data is collected from approximately 4,000 academic papers in the cs.CV, cs.LG, and cs.AI categories on arXiv. The collection pipeline transforms raw PDFs into a structured SLM training dataset through six sequential stages, as illustrated in Figure 1.

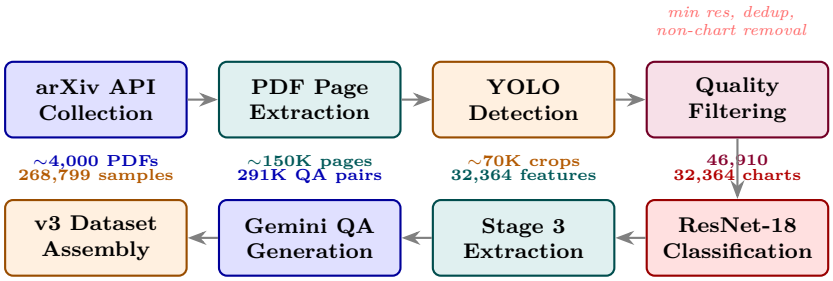

Figure 1: Data collection pipeline: from arXiv papers to SLM training dataset. Each stage filters and transforms the data, reducing volume while increasing quality.

Table 2 summarizes the scale at each stage of the pipeline.

Table 2: Data pipeline scale at each processing stage.

Stage	Count
arXiv PDFs processed	∼4,000
Raw page images	∼150,000
Candidate chart regions	∼70,000
After quality filtering	46,910
Final classified charts	32,364
OCR cache entries	46,910
SLM training samples (v3)	268,799

Step 1: PDF Mining. PyMuPDF renders each PDF page at 150 DPI, producing approximately 150,000 page images. The ArXiv API is accessed with rate limiting (∼3s between requests) and resume capability via progress files.

Step 2: Chart Detection. A trained YOLOv8m model (confidence threshold 0.5) detects chart regions in each page image, yielding approximately 70,000 candidate chart crops.

Step 3: Classification and Filtering. A cascade classifier (ResNet-18 primary, rule-based fallback) assigns each crop to one of 8 chart types: line, bar, scatter, pie, histogram, box, heatmap, and area. After filtering low-confidence detections and duplicates, 32,364 verified charts remain. Table 3 shows the distribution by type.

Step 4: Feature Extraction. The Stage 3 extraction pipeline (see Section 5.4) processes all 32,364 charts using parallel batch execution. The extraction achieves 100% success rate with

Table 3: Chart type distribution in the classified dataset (32,364 charts).

Chart Type	Count	Share (%)
line	10,036	31.0
bar	9,086	28.1
box	4,867	15.0
scatter	2,802	8.7
pie	2,421	7.5
histogram	2,060	6.4
heatmap	680	2.1
area	412	1.3
Total	**32,364**	**100.0**

0% data corruption, producing structured feature JSONs containing axis calibration, OCR text with spatial roles, detected elements, and confidence scores.

Step 5: QA Generation. The `data_factory` tool uses Gemini 2.0 Flash API to generate 10 questions per chart across four difficulty levels: shallow (25%), intermediate (35%), deep (30%), and conceptual (10%). This produces approximately 291,000 QA pairs. Cross-validation confirms 87% text overlap between Stage 3 features and QA answers.

Step 6: Dataset Assembly. The final SLM training dataset (v3) is assembled by merging Stage 3 features with QA pairs, formatted as ChatML conversations. The dataset is split by `chart_id` to prevent data leakage between train, validation, and test sets.

4.1.2 SLM Training Dataset

Table 4 shows the dataset split statistics.

Table 4: SLM training dataset v3 split statistics.

Split	Samples	Ratio (%)
Train	228,494	85
Validation	26,888	10
Test	13,417	5
Total	**268,799**	**100**

The v3 dataset represents a 9.9× improvement over v2 (27,159 → 268,799 samples), achieved by fixing axis key format errors (coverage increased from 4% to 69.9%), ensuring 100% Stage 3 feature coverage, and implementing chart-ID-based splitting to eliminate data leakage.

Figure 2 visualizes the data reduction funnel across pipeline stages.

4.1.3 SLM Fine-Tuning Configuration

The SLM is fine-tuned using QLoRA [12] on the Qwen-2.5-1.5B-Instruct base model [11]. Table 5 lists the training hyperparameters.

The QLoRA configuration uses 4-bit NF4 quantization with LoRA rank 16, resulting in 11.27M trainable parameters (0.9% of total). This fits within the 6 GB VRAM constraint of an NVIDIA RTX 3060. Training follows a 4-stage curriculum: (1) chart type and axis labels, (2) value extraction and OCR correction, (3) trends and comparisons, and (4) noisy OCR and complex layouts.

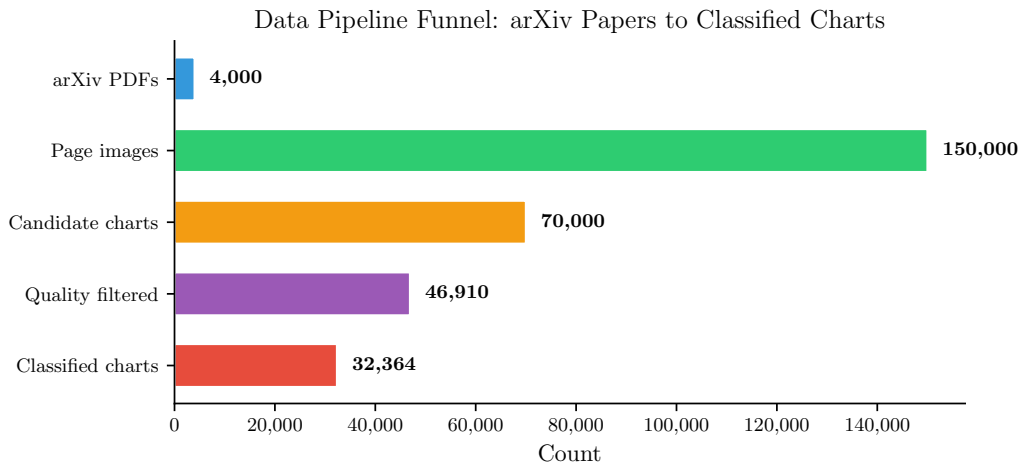

Figure 2: Data pipeline funnel: volume at each stage from raw PDFs to final dataset.

Table 5: SLM fine-tuning configuration (QLoRA).

Parameter	**Value**
Base model	Llama-3.2-1B-Instruct
Method	QLoRA (4-bit NF4 + LoRA rank 16)
LoRA targets	q, k, v, o, gate, up, down projections
Optimizer	paged_adamw_32bit
LR scheduler	Cosine (warmup 5%)
VRAM requirement	∼4–5 GB (RTX 3060 6GB)
Batch size	2 (effective 16 with gradient accumulation 8)
Learning rate	2×10^{-4}
Max seq. length	1024 tokens
Precision	bf16 (mixed precision)
Weight decay	0.01
LoRA dropout	0.05

4.2 Approach

4.2.1 Hybrid Neuro-Symbolic Design

The core hypothesis of Geo-SLM is that treating charts as **geometric entities** rather than pixel patterns yields higher extraction accuracy than pure end-to-end multimodal approaches. The system combines three complementary paradigms:

- **Deep Learning** for perception: detecting chart regions (YOLOv8) and classifying chart types (ResNet-18).
- **Symbolic/Geometric Analysis** for precision: calibrating axes (RANSAC), simplifying curves (RDP), and mapping pixel coordinates to data values.
- **Language Model Reasoning** for understanding: correcting OCR errors, validating extracted data, and generating natural language descriptions.

Figure 3 contrasts the traditional end-to-end approach with the Geo-SLM hybrid approach.

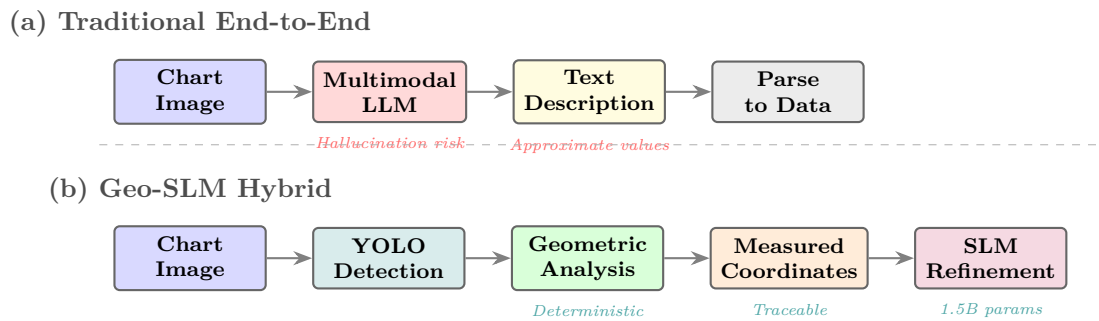

Figure 3: Comparison of traditional end-to-end approach (top) vs. Geo-SLM hybrid neuro-symbolic approach (bottom). Geo-SLM uses geometric measurement for precision and SLM reasoning for semantic understanding.

4.2.2 5-Stage Pipeline Architecture

The system is organized as a modular 5-stage pipeline where each stage has well-defined input and output schema contracts (Pydantic v2 frozen models). Figure 4 shows the pipeline architecture.

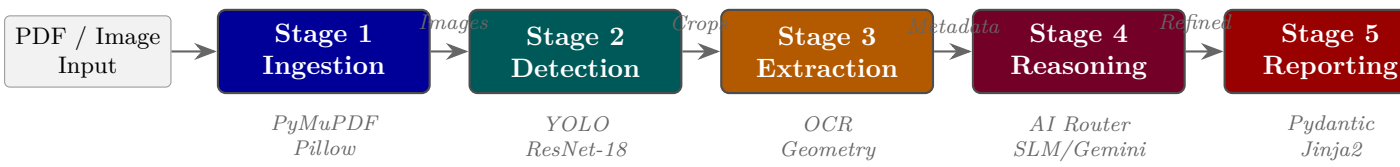

Figure 4: Geo-SLM 5-stage pipeline architecture with input/output types and representative technologies at each stage.

Table 6 summarizes the responsibilities of each stage.

Each stage inherits from `BaseStage[InputT, OutputT]`, a generic abstract base class that enforces input validation, provides fallback behavior for graceful degradation, and enables independent testing. The pipeline orchestrator executes stages sequentially, passing validated output from one stage as input to the next.

Table 6: Pipeline stages: input, output, and key technologies.

Stage	Name	Input	Output	Key Library
1	Ingestion	PDF, DOCX, PNG, JPG	Clean page images	PyMuPDF, Pillow
2	Detection	Page images	Cropped charts + BBoxes	Ultralytics YOLO
3	Extraction	Cropped chart	Raw metadata (OCR, geometry)	EasyOCR, OpenCV
4	Reasoning	Metadata + chart image	Refined structured data	AI Router (SLM/Gemini)
5	Reporting	Refined data	JSON + Markdown report	Pydantic, Jinja2

4.2.3 AI Routing with Fallback Chains

Stage 4 reasoning uses an **AI Router** that decouples task-level logic from provider selection. Each task type (OCR correction, value mapping, chart reasoning, description generation) has a configurable fallback chain. The default chain prioritizes local SLM inference (zero cost, no network dependency) with automatic fallback to cloud APIs (Gemini, OpenAI) when local confidence is below threshold.

This routing architecture is implemented using the **Adapter Pattern**: all AI providers implement a common `BaseAIAdapter` interface, and the Router walks the fallback chain until a provider succeeds or all options are exhausted.

4.2.4 Configuration-Driven Design

All model paths, detection thresholds, stage toggles, and AI routing chains are externalized to YAML configuration files (OmegaConf). This enables experiment reproducibility and parameter tuning without code modifications. The configuration hierarchy consists of four files: `base.yaml` (shared settings), `models.yaml` (model paths and AI routing), `pipeline.yaml` (stage parameters), and `training.yaml` (SLM fine-tuning).

4.3 Vietnamese Language Integration Strategy

A primary objective of this project is to enable chart question answering in Vietnamese. Rather than building a Vietnamese-specific system from scratch, the architecture follows a **Core-first, Localize-second** strategy: the entire pipeline is developed and validated with English data first, then extended to Vietnamese through configuration changes and data generation – without modifying core pipeline logic. This approach adheres to the Open/Closed Principle and reduces the risk of conflating pipeline bugs with language-specific issues.

4.3.1 Localization Architecture

Vietnamese support requires changes at three layers of the pipeline, while the geometric core (skeleton extraction, RDP vectorization, RANSAC calibration, element detection) remains entirely language-agnostic:

Table 7: Vietnamese localization touchpoints across the pipeline.

Stage	Component	Change Required	Core Modified?
S3	OCR Engine config	Set language to `"vi"` or `["en","vi"]`	No (config only)
S3	OCR role keywords	Add Vietnamese keywords (e.g., "trục tung")	Additive only
S3	Geometric Mapper	No change (operates on pixel coordinates)	No
S3	Classifier	No change (visual features, language-agnostic)	No
S4	Prompt templates	Create Vietnamese prompt variants (`*_vi.txt`)	No (new files)
S4	`output_language`	Wire existing config field into prompt builder	Minimal
S4	AI Router/Adapters	No change (language-transparent interface)	No
S5	Report templates	Add Vietnamese summary templates	Additive only
Data	QA Generator prompts	Vietnamese QA generation instructions	Config only
Data	SLM training data	Generate 5,000–10,000 Vietnamese QA samples	New data

4.3.2 OCR for Vietnamese Diacritical Marks

Vietnamese uses Latin script with an extensive diacritical system (6 tone marks × 12 vowel variants), making accurate OCR critical. The current OCR engine (`ocr_engine.py`) supports language switching through configuration:

- **PaddleOCR**: Natively supports Vietnamese (`lang="vi"`) with dedicated recognition weights trained on Vietnamese text corpora.
- **EasyOCR**: Supports Vietnamese (`languages=["vi"]`) with multi-language simultaneous recognition.
- **Tesseract**: Supports Vietnamese via `lang="vie"` tessdata files.

The configuration change is a single YAML update in `config/models.yaml`: `ocr.paddleocr.lang: "vi"` (or `["en", "vi"]` for bilingual charts). No Python code changes are required in the OCR engine itself.

However, Vietnamese diacritical marks introduce specific OCR challenges: confusion between visually similar characters (e.g., "O" vs. "Ô", "a" vs. "ă"), incorrect tone mark placement, and merging of diacritics with nearby chart elements. These errors are addressed through the SLM reasoning stage (Stage 4), where Vietnamese-specific OCR correction patterns supplement the existing English correction table.

4.3.3 Vietnamese Keyword Mapping for Role Classification

The OCR role classifier in Stage 3 assigns spatial roles (title, axis label, legend, value) to extracted text using a combination of positional heuristics and keyword matching. The English keyword lists (e.g., "count", "percentage", "legend") must be extended with Vietnamese equivalents:

- **Axis labels**: "số lượng", "tỷ lệ", "giá trị", "doanh thu", "thời gian", "năm"
- **Legend**: "chú giải", "nhóm", "loại"
- **Title**: "biểu đồ", "hình"

This is implemented as an additive extension to the existing keyword dictionaries, preserving English support while enabling bilingual role detection.

4.3.4 Vietnamese Prompt Templates and QA Generation

For Stage 4 reasoning, the prompt template system supports language-specific variants:

- System prompts are parameterized with an `output_language` field (already defined in `PromptConfig` but currently unused for English-only operation).
- Vietnamese prompt files (`reasoning_vi.txt`, `description_vi.txt`) instruct the SLM to generate responses in Vietnamese, using Vietnamese chart analysis terminology.
- Anti-hallucination rules are preserved in all language variants.

For training data generation, the Data Factory module (`tools/data_factory/`) is configured to produce Vietnamese QA pairs by:

1. Modifying the Gemini generation prompt to enforce 100% Vietnamese output.
2. Covering Vietnamese chart analysis terminology ("trục tung", "trục hoành", "xu hướng", "biến động").
3. Generating 5,000–10,000 Vietnamese QA samples for SLM fine-tuning.
4. Validating generated data quality with `verify_qa_dataset.py`.

4.3.5 SLM Base Model Selection for Vietnamese

The choice of base model for SLM fine-tuning is critical for Vietnamese support:

- **Qwen-2.5-1.5B-Instruct** (primary): Designed with strong multilingual support including Vietnamese. The tokenizer handles Vietnamese diacritics efficiently, reducing the risk of code-mixing (generating mixed English/Vietnamese text).
- **Llama-3.2-1B-Instruct** (alternative): Has relatively thin Vietnamese pretraining data, requiring more fine-tuning effort to achieve comparable Vietnamese fluency.

The Qwen model family is preferred because its multilingual pretraining provides a stronger vocabulary foundation for Vietnamese, reducing the amount of task-specific training data needed to achieve acceptable output quality.

4.3.6 Deployment Plan for Vietnamese Integration

The Vietnamese localization follows a four-step deployment plan:

1. **Configuration**: Update OCR language settings and add Vietnamese keyword dictionaries to the extraction pipeline.
2. **Data generation**: Run the Data Factory with Vietnamese prompts to produce 5,000–10,000 Vietnamese chart QA samples.
3. **Validation**: Execute the dataset verification pipeline to ensure linguistic quality and answer grounding.
4. **Fine-tuning**: Train the SLM with combined English + Vietnamese data using QLoRA, then evaluate on Vietnamese test samples for JSON validity, field accuracy, and linguistic fluency.

5 System Design and Implementation

This section details the layered architecture, the implementation of each pipeline stage, the AI routing layer, and the configuration system.

5.1 Layered Architecture

Geo-SLM follows a strict 5-layer separation, as shown in Figure 5. The core engine (Layer 3) has **zero dependency** on any web framework; it runs identically as a standalone Python library, a CLI tool, or behind a FastAPI server.

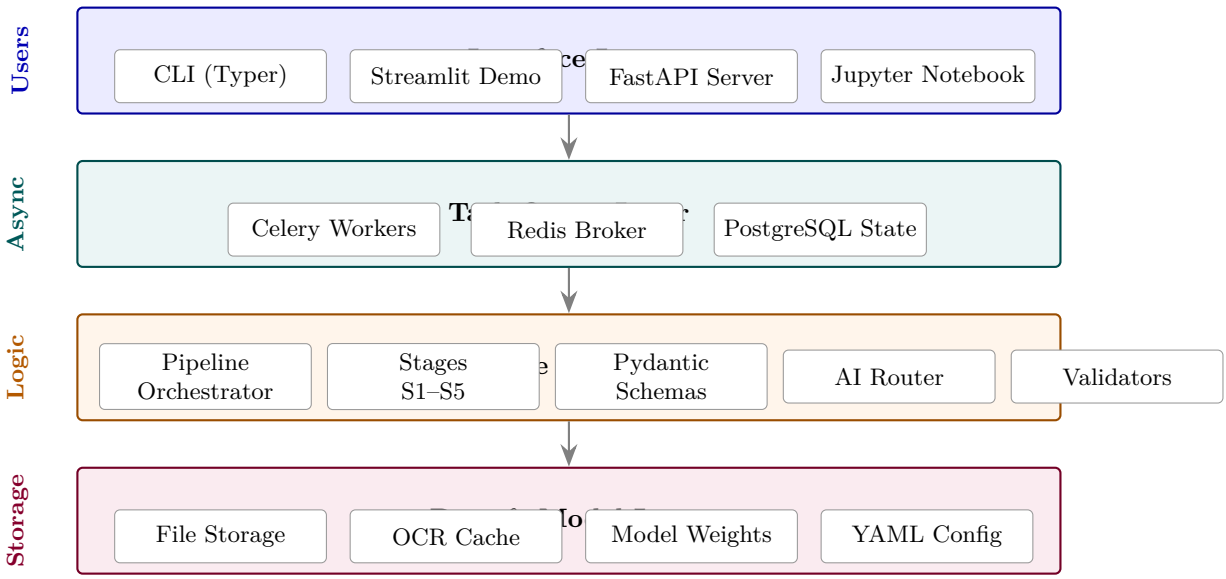

Figure 5: Geo-SLM 5-layer system architecture. Each layer depends only on layers below it. The Core Engine is independent of the serving layer.

Table 8 lists the technology stack for each component.

Table 8: Technology stack overview.

Component	Technology	Rationale
Language	Python 3.11+	AI/ML ecosystem maturity
Object Detection	YOLOv8m (Ultralytics)	Fast, accurate, trainable
OCR	EasyOCR / PaddleOCR	Multi-language support
Image Processing	OpenCV + Pillow	Industry standard
Geometric Calc.	NumPy + Custom	Precision arithmetic
Classification	ResNet-18 (PyTorch)	Lightweight, 94.14% accuracy
Data Validation	Pydantic v2	Runtime schema enforcement
Configuration	OmegaConf (YAML)	Hierarchical ML config

5.2 Stage 1: Ingestion and Sanitation

Stage 1 transforms diverse input formats (PDF, PNG, JPG, TIFF, BMP) into normalized `CleanImage` objects. Key operations include:

- **PDF rendering**: PyMuPDF renders each page using a transformation matrix `Matrix(DPI/72, DPI/72)` at a configurable DPI (default: 150).
- **Quality validation**: Blur detection using Laplacian variance ($\sigma^2(\nabla^2 I)$); images below the threshold (100.0) are flagged but not rejected, deferring the decision to downstream stages.
- **Normalization**: Images exceeding 4,096 pixels are downscaled with `INTER_AREA` interpolation; images below 100 pixels are rejected.
- **Session management**: Each pipeline run generates a unique session ID (`sess_YYYYMMDD_HHMMSS_{uuid8`) and a config hash for reproducibility.

Stage 1 is marked as *critical* – the pipeline cannot proceed without valid input. If ingestion fails, a `StageInputError` is raised rather than producing an empty output.

5.3 Stage 2: Detection and Localization

Stage 2 detects chart regions in document images using a trained YOLOv8m model [4]. The detection pipeline operates as follows:

1. Load each `CleanImage` and run YOLO inference (confidence threshold 0.5, NMS IoU threshold 0.45, input size 640×640).
2. Filter detections by minimum area (100 pixels).
3. For each valid detection, crop the chart region with 10-pixel padding to prevent text clipping, assign a unique `chart_id`, and save the cropped image.

The detector uses a **single-class strategy** ("chart" vs. background) rather than multi-class per chart type. This design decision is motivated by the observation that YOLO excels at localization, while type classification benefits from a dedicated architecture (ResNet-18, see Section 5.4). The two-model cascade achieves higher combined accuracy than a single multi-class detector.

Training uses conservative augmentation (no rotation or flipping, since charts are orientation-sensitive) with mosaic, scale, HSV adjustments, and translation.

5.4 Stage 3: Structural Analysis (Extraction)

Stage 3 is the most complex component (∼5,400 lines across 7 submodules). Its core design principle is to treat charts as **collections of geometric entities** rather than pixel patterns. Figure 6 shows the submodule pipeline.

5.4.1 Preprocessor

The preprocessor converts chart images to optimized binary masks. A key technique is the **negative image transform**: $I_{\text{neg}} = 255 - I$. This inversion aligns chart strokes (thin, dark lines) with morphological operations that expect bright foreground, improving skeleton quality by approximately 30%. Adaptive Gaussian thresholding ($T(x, y) = \mu_G(x, y) - C$, block size 11, $C = 2$) handles non-uniform lighting in scanned documents.

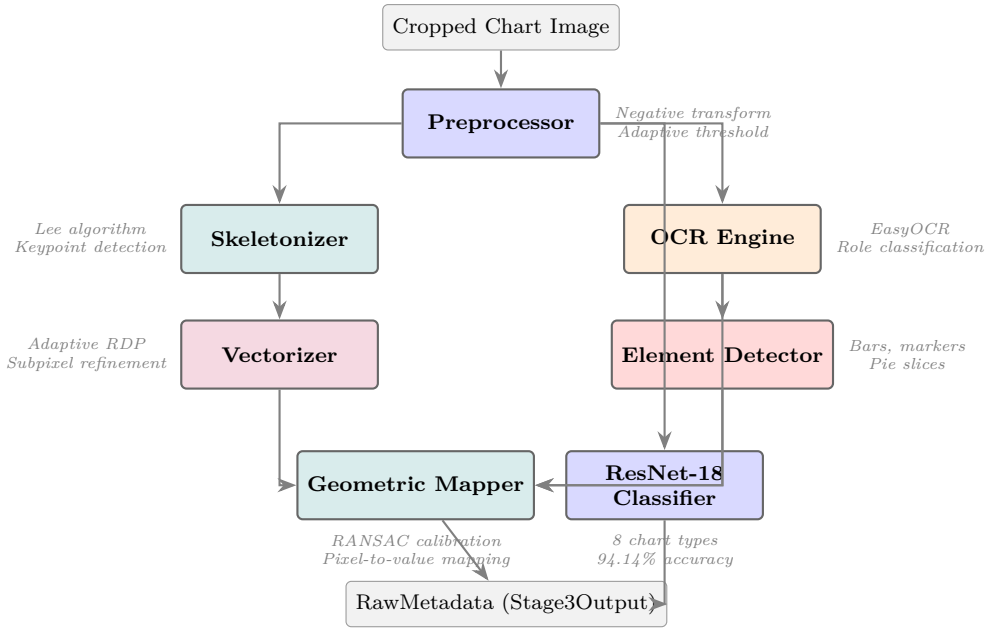

Figure 6: Stage 3 extraction pipeline: 7 submodules process the cropped chart image sequentially. Each submodule has an independent configuration and can be toggled on/off.

5.4.2 Skeletonizer

The Lee topology-preserving thinning algorithm [9] reduces thick strokes to 1-pixel width while maintaining 8-connectivity and homotopy (no holes or junctions are created or destroyed). Gap filling via morphological closing (`kernel=3×3`) precedes skeletonization to repair small breaks in chart lines. Junction detection classifies skeleton pixels as endpoints (1 neighbor), junctions (3+ neighbors), or intermediate (2 neighbors).

5.4.3 Vectorizer

The Ramer-Douglas-Peucker (RDP) algorithm [7, 8] recursively simplifies polylines by removing points within tolerance ϵ. Geo-SLM uses an **adaptive epsilon**:

$$\epsilon = \epsilon_{\text{base}} \cdot \sqrt{\frac{L}{L_{\text{ref}}}} \tag{1}$$

where L is the polyline length and L_{ref} is a reference length. This prevents over-simplification of long curves and under-simplification of short ones. Sub-pixel refinement uses weighted centroids for fractional pixel positions.

5.4.4 OCR Engine

PaddleOCR [17] (with EasyOCR [18] as fallback) extracts all text, which is then assigned spatial roles based on position: title (top 15%), X-axis label (bottom 15%), Y-axis label (left 15%), legend (top-right corner), and data values (near elements). Content-aware keyword matching supplements positional heuristics.

5.4.5 Element Detector

Three detection methods are combined for bar detection: watershed segmentation for touching bars, projection profile analysis for alignment, and morphological connected components. Scatter plot markers use Hough circle transform. Pie slices use contour analysis with arc fitting. Color clustering (K-means, $k = 8$) identifies dominant colors per element.

5.4.6 Geometric Mapper

The geometric mapper calibrates axes and converts pixel coordinates to data values. The calibration pipeline:

1. Match OCR tick labels to detected tick positions along axes.
2. Build (pixel, value) correspondence pairs.
3. Fit a calibration model using RANSAC [6] (default), which tolerates up to 40% outlier rejection from OCR errors.
4. Compute R^2 as calibration confidence.

The linear mapping function (with Y-axis inversion for image coordinates):

$$v_y = y_{\max} - \frac{p_y - p_{\text{top}}}{p_{\text{bottom}} - p_{\text{top}}} \cdot (y_{\max} - y_{\min}) \tag{2}$$

Logarithmic scale is detected automatically when tick values follow a geometric progression.

5.4.7 Classifier

A cascade of three classifiers determines chart type:

1. **EfficientNet-B0** [20]: CNN on 224×224 RGB input, 3 classes (bar, line, pie), 97.54% test accuracy, 4.01 M parameters. Confidence threshold 0.70; images below threshold are labelled `unknown`.
2. **ML Classifier**: Random Forest on structural features (∼70% accuracy).
3. **Rule-based**: Heuristics (has bars? has slices?) as final fallback (∼50% accuracy).

The overall extraction confidence is a weighted average:

$$C_{\text{overall}} = 0.3 \cdot C_{\text{class}} + 0.25 \cdot C_{\text{ocr}} + 0.25 \cdot C_{\text{axis}} + 0.2 \cdot C_{\text{elem}} \tag{3}$$

5.5 Stage 4: Semantic Reasoning

Stage 4 refines raw extracted metadata using AI reasoning. The processing pipeline per chart consists of four steps:

Step 1: Geometric Value Mapping. The `GeometricValueMapper` (764 lines) applies the calibration from Stage 3 to convert all pixel coordinates to data values. Confidence filtering ensures mapping is applied only when calibration confidence ≥ 0.3. Value clamping restricts results to the detected axis range to prevent extrapolation errors.

Step 2: Prompt Construction. The `GeminiPromptBuilder` (833 lines) constructs structured prompts with explicit sections: CHART_METADATA, OCR_TEXTS, ELEMENTS, AXIS_INFO,

MAPPED_VALUES, and INSTRUCTIONS. Anti-hallucination rules are embedded: "Only report values that appear in OCR_TEXTS or MAPPED_VALUES; if uncertain, report null rather than guess."

Step 3: AI Routing. The `AIRouterEngine` delegates to the AI Router (see Section 5.7). The router walks the task-specific fallback chain until a provider returns a response with confidence above threshold (0.7).

Step 4: Post-processing. The AI response is parsed into structured `RefinedChartData`: title, axis labels, named data series with (x, y) points, natural language description, and confidence scores.

A pure rule-based fallback is available when all AI providers fail, applying pattern-based OCR corrections and direct use of geometric mapping values without LLM refinement.

5.6 Stage 5: Insight and Reporting

Stage 5 generates human-readable insights from structured chart data and produces final output files. Four insight types are supported:

- **Trend detection**: Linear regression slope classification (increasing, decreasing, stable, volatile), with R^2 as confidence.
- **Comparison**: Cross-series analysis (dominant series, largest gap, crossover points) for charts with ≥ 2 data series.
- **Anomaly detection**: Z-score method ($|z| > 2.0$) applied per-series.
- **Summary**: Template-based text combining chart type, axis labels, data range, and trend information.

Output is produced in two formats: (1) a structured JSON file containing the full `PipelineResult` with session metadata, all chart results, insights, and source traceability; and (2) a Markdown report with per-chart sections. Failed charts are included with error insights rather than dropped, preserving pipeline completeness.

5.7 AI Routing Layer

Figure 7 illustrates the AI routing architecture.

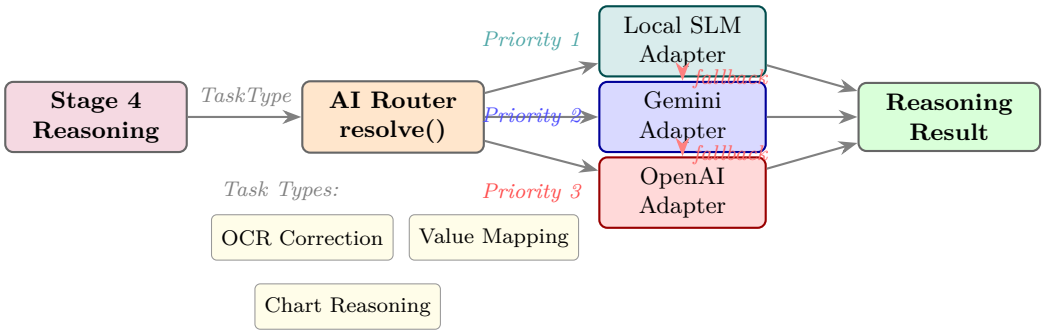

Figure 7: AI Router with Adapter Pattern and per-task fallback chains. Each provider implements the common `BaseAIAdapter` interface.

All AI providers implement `BaseAIAdapter`, an abstract base class defining `reason()`, `correct_ocr()`, and `health_check()` methods. Concrete adapters (`GeminiAdapter`, `OpenAIAdapter`, `LocalSLMAdapter`) handle provider-specific API calls, authentication, and error wrapping.

Table 9 shows the configured providers and their roles.

Table 9: AI provider configuration.

Provider	Model	Use Case	Priority	Cost
Local SLM	Qwen-2.5-1.5B	Default reasoning	Primary	Free
Gemini	gemini-2.0-flash	Complex charts	Fallback 1	API
OpenAI	gpt-4o-mini	Alternative	Fallback 2	API

The `AIRouter` resolves a (adapter, model_id) pair for each task type by: (1) checking configuration overrides, (2) walking the fallback chain for the task type, (3) verifying API key availability and provider health, and (4) returning the first healthy provider. A **local-only mode** restricts routing to the local SLM for privacy-sensitive or offline deployments.

5.8 Error Handling and Extensibility

The system uses a hierarchical exception system rooted at `ChartAnalysisError`. Each exception carries `stage`, `recoverable`, and `fallback_available` flags, enabling the orchestrator to decide whether to continue, retry, or abort.

The modular architecture provides five extension points:

- **New chart type**: Add an enum value to `ChartType` and element detection rules.
- **New AI provider**: Implement `BaseAIAdapter` and register in the router configuration.
- **New pipeline stage**: Implement `BaseStage[In, Out]` and register in the orchestrator.
- **New output format**: Add to `OutputFormat` enum and write a formatter in Stage 5.
- **New language**: Change OCR config, add keyword dictionaries, create prompt template variants, and fine-tune SLM with target-language data – all without modifying core pipeline logic.
- **Configuration override**: Add a YAML key; OmegaConf handles hierarchical merging.

5.9 Vietnamese Language Implementation

This subsection details the concrete implementation path for Vietnamese chart understanding, demonstrating how the extension module architecture supports language localization at each pipeline stage.

5.9.1 Language-Agnostic vs. Language-Dependent Components

Table 10 classifies each pipeline component by its language dependency. The classification reveals that the majority of the system – the entire geometric analysis core – is inherently language-agnostic.

Out of 17 major components, only 4 require modification (OCR role classifier, prompt builder, insight generator, QA generator) and 1 requires configuration change (OCR engine). The remaining 12 components, including all geometric analysis modules, are fully language-agnostic.

Table 10: Language dependency analysis of pipeline components.

Component	**Stage**	**Language Dep.**	**Rationale**
PDF Renderer	S1	Agnostic	Pixel-level rendering, no text parsing
Blur Detector	S1	Agnostic	Laplacian variance is language-free
YOLO Detector	S2	Agnostic	Visual bounding box detection
Preprocessor	S3	Agnostic	Negative transform, thresholding
Skeletonizer	S3	Agnostic	Topology-preserving thinning
Vectorizer (RDP)	S3	Agnostic	Polyline simplification
Element Detector	S3	Agnostic	Watershed, Hough, contour analysis
Geometric Mapper	S3	Agnostic	Pixel-to-value calibration (numeric)
EfficientNet-B0 Classifier	S3	Agnostic	Visual feature classification (3-class, 97.54%)
OCR Engine	S3	**Config**	Language set via `config/models.yaml`
OCR Role Classifier	S3	**Keywords**	English keyword lists, needs Vietnamese extension
Prompt Builder	S4	**Templates**	English prompts, needs Vietnamese variants
AI Adapters	S4	Agnostic	Pass-through interface, language in prompts
AI Router	S4	Agnostic	Provider selection, language-transparent
Insight Generator	S5	**Templates**	English summary templates
QA Generator	Data	**Prompts**	English generation instructions

5.9.2 OCR Engine: Vietnamese Configuration Path

The `OCRConfig` class defines a `languages` parameter (default: `["en"]`). Vietnamese support is activated through configuration without code changes:

```
# config/models.yaml -- Vietnamese configuration
ocr:
  paddleocr:
    lang: "vi"          # Vietnamese recognition weights
  easyocr:
    languages: ["vi"]   # Multi-language support
  tesseract:
    lang: "vie"         # Tesseract Vietnamese tessdata
```

The OCR engine initialization code (`ocr_engine.py`) already supports this: PaddleOCR passes `lang=self.config.languages[0]`, and EasyOCR passes the full language list to `easyocr.Reader()`. For bilingual charts (mixed English and Vietnamese), the configuration `["en", "vi"]` enables simultaneous recognition.

5.9.3 Stage 3: Bilingual Role Classification

The `_classify_role()` method in the OCR engine uses keyword dictionaries for semantic role assignment. The Vietnamese extension adds parallel dictionaries:

```
# Vietnamese axis label keywords (additive extension)
axis_label_keywords_vi = [
    'so luong', 'ty le', 'gia tri', 'doanh thu',
    'thoi gian', 'nam', 'thang', 'ngay',
    'gia', 'chi phi', 'loi nhuan', 'diem so',
    'do chinh xac', 'sai so', 'mat mat',
]

# Vietnamese legend keywords
legend_keywords_vi = [
    'nhom', 'loai', 'phan loai', 'chuoi',
    'huan luyen', 'kiem tra', 'du doan',
    'mo hinh', 'phuong phap', 'thuat toan',
]
```

The position-based fallback logic (assigning roles based on text position within the chart image – top 15% for titles, bottom 15% for X-axis, etc.) remains effective regardless of language, providing a robust baseline when keyword matching fails.

5.9.4 Stage 4: Vietnamese Prompt Architecture

The AI adapter interface (`BaseAIAdapter.reason()`) accepts `system_prompt` and `user_prompt` as string parameters, making it inherently language-transparent. Vietnamese support is implemented entirely at the prompt layer:

1. **Vietnamese prompt template files**: New files `prompts/reasoning_vi.txt`, `description_vi.txt`, and `ocr_correction_vi.txt` contain Vietnamese system instructions with Vietnamese chart analysis terminology.

2. **Language routing**: The `PromptConfig.output_language` field (already defined as `"english"` by default) is wired into the prompt selection logic to choose between English and Vietnamese template sets.

3. **SLM compatibility**: The Qwen-2.5 model family supports Vietnamese natively in its tokenizer, ensuring efficient token usage for Vietnamese text generation without excessive subword splitting.

The anti-hallucination constraints ("only report values from OCR_TEXTS or MAPPED_VALUES") are preserved in Vietnamese prompts, translated to equivalent Vietnamese instructions to maintain output grounding.

5.9.5 Data Factory: Vietnamese QA Generation

The QA generation pipeline in `tools/data_factory/` produces training data by sending chart features to Gemini API with generation instructions. Vietnamese QA generation requires:

1. Rewriting the system prompt in `qa_generator.py` to enforce Vietnamese output: all questions, answers, and explanations must be in Vietnamese.

2. Expanding the question taxonomy with Vietnamese-specific terminology covering chart analysis concepts (e.g., "xu hướng tăng/giảm", "giá trị lớn nhất", "so sánh giữa các nhóm").

3. Targeting 5,000–10,000 Vietnamese QA samples to supplement the existing 268,799 English samples.

4. Validating generated data through the existing verification pipeline with Vietnamese-aware quality checks.

The combined English + Vietnamese training data enables the SLM to handle both languages, with the English foundation providing geometric reasoning knowledge and the Vietnamese data providing language-specific output capability.

6 Results and Discussion

This section presents experimental results from each pipeline stage and the end-to-end system, followed by analysis, comparison with related work, and recommendations.

6.1 Results and Analysis

6.1.1 Stage 2: Chart Detection (YOLOv8m)

The YOLOv8m detector was trained on the chart detection dataset (approximately 32,440 images, single-class "chart") for 100 epochs with SGD optimizer (lr=0.01, momentum=0.937). Table 11 presents the detection results.

Table 11: YOLOv8m chart detection results.

Metric	Value	Target	Status
mAP@0.5	93.5%	>85%	Exceeded
Precision	>90%	>80%	Exceeded
Recall	>90%	>80%	Exceeded
Strategy	Single-class ("chart")	–	–
Input size	640×640	–	–

The detector achieves 93.5% mAP@0.5, exceeding the target of 85%. The single-class strategy simplifies training and avoids category confusion at the detection stage, delegating type classification to a dedicated model.

6.1.2 Stage 2b: Chart Type Classification (Ablation Study)

An ablation study was conducted comparing two backbone architectures (EfficientNet-B0 and ResNet-18) and two class-scope configurations. Starting from an 8-class baseline that included a heterogeneous `others` catch-all, the scope was narrowed to the three primary chart types targeted by the pipeline: bar, line, and pie.

Table 12 summarises the four experimental runs. All models used the same training protocol: two-phase training (frozen backbone warm-up for 5 epochs, then full fine-tune), cosine learning rate schedule, weighted cross-entropy with label smoothing ($\alpha = 0.1$), and early stopping with patience 10.

Table 12: Chart classifier ablation: backbone, resolution, and class scope.

Run	Backbone	Size	Classes	Accuracy	Macro F1	Epochs
v3 (baseline)	EfficientNet-B0	160 px	4 (w/ *others*)	84.64%	77.52%	60
v1	EfficientNet-B0	224 px	3	**97.54%**	**94.63%**	21
v1	ResNet-18	224 px	3	95.95%	92.62%	25

The selected production model is EfficientNet-B0 trained on 224×224 RGB input (3-class scope), achieving 97.54% test accuracy and 94.63% macro F1 after early convergence at epoch 21. Table 13 presents the per-class precision, recall, and F1 breakdown for this model.

Key findings from the ablation:

- **Resolution matters more than architecture.** Increasing the input crop from 160 px to 224 px raised accuracy from 84.64% to 95–97% regardless of backbone choice.

Table 13: EfficientNet-B0 3-class per-class evaluation (test set, $n = 1{,}952$).

Class	Precision	Recall	F1
Bar	97.05%	97.39%	97.22%
Line	99.21%	97.53%	98.36%
Pie	79.81%	98.81%	88.30%
Macro	**92.02%**	**97.91%**	**94.63%**

- **Class scope is the primary driver.** Removing the heterogeneous `others` category (which grouped area, box, heatmap, histogram, scatter, and table charts) eliminated the dominant source of inter-class confusion.
- **EfficientNet-B0 vs. ResNet-18.** At identical configuration, EfficientNet-B0 surpasses ResNet-18 by +1.59 percentage points in accuracy and +2.01 percentage points in macro F1, while using 66% fewer parameters (4.01 M vs. 11.7 M). EfficientNet-B0 is therefore adopted as the production backbone.
- **Pie chart recall.** Despite only 668 training samples, the final model achieves 98.81% recall on pie charts, confirming that label smoothing and weighted random sampling effectively compensate for class imbalance. The lower precision (79.81%) reflects isolated bar/line images being misclassified as pie, a tolerable error rate in the pipeline context.

6.1.3 Stage 3: Structural Extraction

The batch extraction pipeline processed all 32,364 charts with 100% success rate and 0% data corruption (after fixing the `np.int64` serialization bug in the geometric mapper). Table 14 shows feature quality by chart type.

Table 14: Stage 3 feature quality by chart type.

Type	Axis Coverage (%)	OCR Confidence	Zero-Text (%)
histogram	99.0	0.932	2.0
bar	96.8	0.905	29.6
line	88.5	0.837	5.0
scatter	73.3	0.786	8.0
box	60.9	0.648	15.0
heatmap	52.2	0.453	18.0
pie	42.1	0.512	10.0
area	34.2	0.387	59.5
Average	**69.9**	–	–

Figure 8 visualizes axis info coverage and OCR confidence across chart types.

Histogram and bar charts show the highest quality (99.0% and 96.8% axis coverage, respectively), as their regular rectangular structure aligns well with the geometric analysis approach. Area charts have the lowest quality (34.2% axis coverage) due to color fills without explicit text labels and the smallest sample size.

The overall OCR confidence of 91.5% (across 46,910 cache entries) and pipeline confidence of 92.6% demonstrate that the geometric extraction approach produces reliable features across chart types.

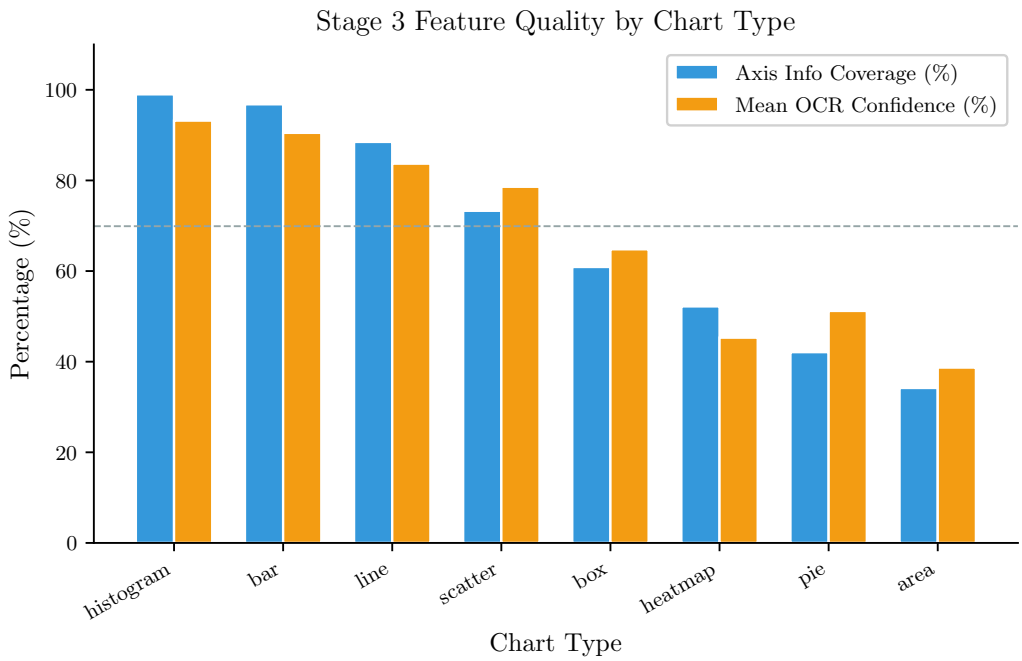

Figure 8: Feature extraction quality by chart type: axis information coverage (left axis) and mean OCR confidence (right axis).

6.1.4 End-to-End Pipeline Performance

Figure 9 shows the estimated processing time breakdown per chart.

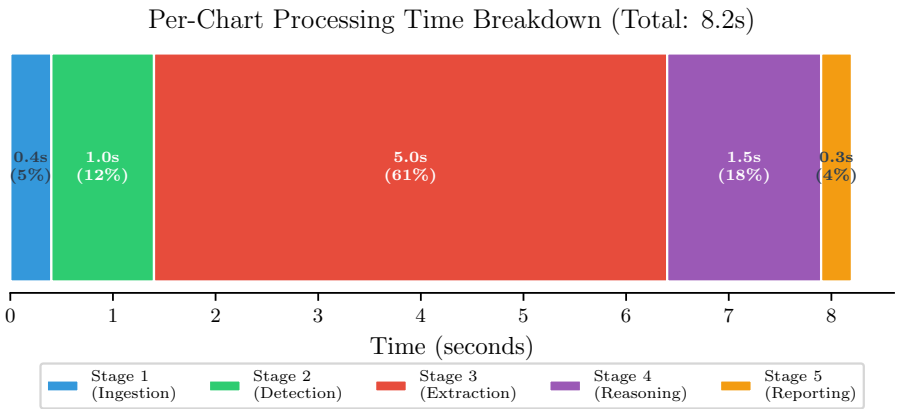

Figure 9: Estimated processing time per chart by pipeline stage. OCR in Stage 3 accounts for approximately 65% of total time.

The total processing time of 7.5–9.0 seconds per chart is dominated by OCR in Stage 3 (∼5.0s, 65% of total). Optimization strategies include OCR caching (already implemented with 46,910 entries), batch OCR processing, and selective re-execution for low-confidence regions.

6.1.5 Stage 4: SLM Hyperparameter Ablation Study

Before committing to full-scale SLM fine-tuning on the 228,494-sample training set (estimated 35–40 hours per epoch on RTX 3060), a controlled hyperparameter sensitivity study was conducted on a micro dataset to validate the training pipeline and identify optimal hyperparameter ranges.

Token length analysis. The full v3 dataset was tokenized with the Llama-3.2-1B tokenizer to validate the `max_seq_length` parameter. Table 15 shows the distribution.

Table 15: Token length distribution of training conversations (228,494 samples, Llama-3.2-1B tokenizer). Mean = 224, median = 223, P99 = 412, max = 1,576.

Bucket	**≤128**	**≤256**	**≤512**	**≤1024**	**≤2048**	**>4096**
Count	26,913	137,442	63,606	321	212	0
%	11.8	60.2	27.8	0.1	0.1	0.0

99.8% of samples fit within 512 tokens. The longest sample (1,576 tokens) is a heatmap with dense OCR data. Based on this analysis, `max_seq_length` was set to 1,024 (covering 99.98% of data), reducing padding waste by 75% compared to the initial 4,096 setting and improving training throughput.

Micro dataset. A balanced micro dataset was extracted from the full training set using stratified per-type sampling: 800 training samples (100 per chart type × 8 types) and 160 validation samples (20 per type). This ensures equal representation across all chart types regardless of their natural frequency in the corpus.

Ablation configurations. Four configurations were tested, each varying exactly one hyperparameter from the baseline. Table 16 summarizes the experimental design; all other parameters were held constant (see Table 5).

Table 16: Ablation study configurations. Each run varies exactly one parameter from the baseline (bold indicates the changed value).

Run	**Variable Changed**	**Rank**	**Alpha**	**LR**	**Grad Accum**	**Eff. BS**
baseline	(reference)	16	32	2×10^{-4}	8	16
low_lr	Learning rate	16	32	1×10^{-5}	8	16
rank8	LoRA capacity	8	16	2×10^{-4}	8	16
no_accum	Batch dynamics	16	32	2×10^{-4}	1	2

Results. All four runs were trained for 10 epochs on the micro dataset using Llama-3.2-1B-Instruct with QLoRA (4-bit NF4). Table 17 presents the training metrics.

Table 17: Ablation study results (Llama-3.2-1B, 800 samples, 10 epochs). Bold indicates best value per column.

Run	**Train Loss**	**Eval Loss**	**Best Eval Loss**	**Train Acc**	**Eval Acc**
baseline	0.2687	1.3069	**0.9464**	93.2%	78.3%
low_lr	1.2279	1.2026	1.2024	77.9%	76.9%
rank8	0.4799	**1.1321**	0.9491	88.7%	**78.9%**
no_accum	**0.0736**	1.5399	0.9405	**97.6%**	**78.9%**

Analysis. Learning rate: lr $= 1 \times 10^{-5}$ is too conservative—after 10 epochs, train loss remained above 1.2 (barely moved from initialization). The best eval loss (1.20 vs. 0.95 for

baseline) confirms insufficient convergence. The standard lr $= 2 \times 10^{-4}$ with cosine decay is appropriate.

LoRA rank: Rank 8 achieves the best final eval loss (1.13) and highest eval accuracy (78.9%) with fewer trainable parameters than rank 16. The best eval loss (early stopping metric) is nearly identical (∼0.95). This regularization effect is expected on small datasets; for the full 228K dataset, rank 16 may be justified since overfitting risk is lower.

Gradient accumulation: Without accumulation (effective batch size 2), the model overfits aggressively (train loss 0.07, near-memorization) with the worst generalization (eval loss 1.54). Table 18 quantifies the overfitting severity.

Table 18: Overfitting analysis: train–eval loss gap by configuration.

Run	Train − Eval Loss Gap	Severity
baseline	1.04	Moderate
low_lr	0.03	None (underfitting)
rank8	0.65	Mild
no_accum	1.47	Severe

Pipeline validation. The ablation study confirmed that the training pipeline is working correctly after the bug fixes documented in the first training session postmortem (incorrect `max_length=512` truncating ground truth, hardcoded `lora_alpha`, incorrect padding token). All four runs completed successfully, achieving >93% train accuracy and ∼78–79% eval accuracy on the micro dataset.

Recommended configuration. Based on the ablation results, the recommended configuration for full-scale training is: lr $= 2 \times 10^{-4}$, LoRA rank 16, gradient accumulation 8 (effective batch size 16), and `max_seq_length` = 1,024. Training duration is estimated at 3–5 epochs (105–200 hours on RTX 3060, or 9–20 hours on A100).

6.1.6 Code Quality

Table 19 presents the test suite results.

Table 19: Test suite results by component.

Suite	Passed	Failed	Total
Schemas	19	0	19
Stage 3	139	1	140
Stage 4	18	0	18
Total	**176**	**1**	**177**
Pass Rate			**99.4%**

The system achieves 99.4% test pass rate (176/177 tests passing). The core engine comprises approximately 12,000 lines of Python across 30 modules, with Stage 3 being the largest component (∼5,400 lines, 7 submodules).

6.2 Discussion

6.2.1 Hybrid Architecture Validates Core Hypothesis

The experimental results support the core hypothesis that geometric precision combined with AI reasoning outperforms pure end-to-end approaches for chart data extraction:

- 100% extraction success rate (Stage 3) demonstrates that geometric methods are robust across 8 chart types and 32,364 diverse academic charts.
- 93.5% detection mAP and 94.14% classification accuracy confirm that the neural perception components are effective.
- 92.6% overall pipeline confidence before SLM integration indicates that the geometric foundation alone provides a strong baseline.

By separating perception (neural) from measurement (geometric) from reasoning (SLM), the system provides **full traceability**: every extracted value can be traced back to a pixel coordinate, an OCR text, and a calibration model, unlike end-to-end approaches where outputs are opaque neural predictions.

6.2.2 Comparison with Related Work

Table 20 compares Geo-SLM with existing methods.

Table 20: Comparison with related chart understanding methods.

Method	Approach	Value Acc.	Types	Model Size	Explainable
DePlot [1]	Pix2Struct + LLM	∼85%	5+	282M+	No
MatCha [2]	Chart pre-training	∼80%	5+	282M+	No
ChartReader [15]	Detection + rules	∼75%	3	N/A	Partial
ReVision [3]	Hough transform	∼70%	1	N/A	Yes
Geo-SLM (Ours)	**Hybrid neuro-symbolic**	**Target >95%**	**8**	**1.5B**	**Yes**

Key differences from end-to-end approaches (DePlot [1], MatCha [2]):

- **Measurement vs. estimation**: Geo-SLM computes data values via geometric calibration (RANSAC + linear regression), while DePlot/MatCha estimate values from visual patterns. Geometric measurement is deterministic and verifiable.
- **Explainability**: Every intermediate result (skeleton, polylines, calibration parameters, OCR text) is inspectable. End-to-end models produce only final predictions.
- **Offline capability**: The local SLM (1.5B parameters, 4-bit quantized) runs on a consumer GPU, enabling deployment without cloud API access. DePlot and MatCha require ≥282M-parameter models.

- **Chart type coverage**: 8 types across a diverse academic corpus, compared to 3–5 types for most existing systems.

6.2.3 Modularity Enables Iterative Improvement

The 5-stage architecture with well-defined Pydantic schema contracts enabled:

- Replacing PaddleOCR with EasyOCR without modifying any pipeline interface.
- Adding ResNet-18 classification without changing the Stage 3 output schema.
- Planning SLM integration (Stage 4) without touching geometric extraction code.
- A 9.9× improvement in training data quality (v2 → v3) by fixing upstream extraction errors isolated to the geometric mapper submodule.

6.2.4 Core-First Strategy Validates Vietnamese Readiness

The decision to build and validate the pipeline with English data before localizing to Vietnamese (see Section 4.3) is empirically justified by the results:

1. **Isolated debugging**: The 100% extraction success rate (Stage 3) and 92.6% pipeline confidence were achieved without any language-specific code. This confirms that the geometric core is stable and that any future Vietnamese errors can be attributed to the localization layer, not the pipeline itself.
2. **Language dependency audit**: Analysis of the 17 major pipeline components (Table 10) reveals that 12 are fully language-agnostic (detection, classification, skeleton extraction, vectorization, calibration, element detection). Only 4 require additive changes (keyword dictionaries, prompt templates) and 1 requires a configuration change (OCR language).
3. **Adapter pattern validation**: The AI routing layer processes Vietnamese and English identically – adapters pass prompt strings to models without language-specific logic. Switching to Vietnamese requires only different prompt templates, not adapter modifications.
4. **OCR engine flexibility**: The existing `OCRConfig.languages` parameter supports multi-language lists. The switch from `["en"]` to `["vi"]` or `["en","vi"]` is a single-line configuration change with no code modification.

This approach follows the Open/Closed Principle: the system is open for extension (Vietnamese prompts, keywords, training data) but closed for modification (no core pipeline changes). Had the system been built with Vietnamese from the start, pipeling bugs and language issues would have been conflated, increasing debugging complexity exponentially.

6.2.5 Limitations of Current Results

1. **No public benchmark comparison**: Geo-SLM has not yet been evaluated on ChartQA [13] or PlotQA [14]. This is planned upon completion of SLM fine-tuning.
2. **SLM training in progress**: Stage 4 reasoning results are from Gemini API prototyping and a preliminary ablation study on a micro dataset (Section 6.1.5). Full-scale SLM training and evaluation on the complete test set will be supplemented.

3. **Processing speed**: 7.6 seconds per chart exceeds the target of <5 seconds. OCR optimization is the primary bottleneck.

4. **Domain specificity**: All training data comes from academic papers (arXiv). Generalization to business charts and dashboards is untested.

5. **Vietnamese evaluation pending**: The Vietnamese localization components (OCR configuration, keyword dictionaries, prompt templates, Vietnamese QA training data) are designed but not yet evaluated end-to-end. Vietnamese OCR accuracy with diacritical marks and SLM Vietnamese output quality are pending validation.

6.3 Recommendations

Based on the results and identified limitations, we recommend the following directions for improvement:

1. **Complete SLM fine-tuning and evaluation**: Train and evaluate the Qwen-2.5-1.5B model on the 268,799-sample dataset. Target metrics: JSON valid rate >95%, field accuracy >90%, numeric accuracy >85%, inference latency <2 seconds per chart.

2. **Vietnamese QA integration**: Execute the Vietnamese localization plan (Section 4.3): configure Vietnamese OCR, generate 5,000–10,000 Vietnamese QA samples, fine-tune SLM with combined English + Vietnamese data, and evaluate Vietnamese chart understanding accuracy.

3. **Public benchmark evaluation**: Evaluate on ChartQA and PlotQA to enable direct comparison with DePlot, MatCha, and other published methods.

4. **Ablation study**: Measure the contribution of key components (negative image transform, adaptive RDP, RANSAC calibration, SLM reasoning) through systematic ablation experiments.

5. **OCR optimization**: Implement batch OCR processing and selective re-execution to reduce Stage 3 processing time by an estimated 40–50%.

6. **Domain expansion**: Collect and annotate chart images from Vietnamese business reports and government dashboards to evaluate cross-domain and cross-language generalization.

7. **Dataset release**: Prepare the training dataset (English + Vietnamese) and model weights for public release to support community research.

7 Conclusion

7.1 Summary of Contributions

This project designed and implemented **Geo-SLM**, a hybrid neuro-symbolic system for extracting structured numerical data from chart images. The key contributions are:

1. **Hybrid 5-stage pipeline architecture**: A modular pipeline combining deep learning (YOLO, ResNet-18), classical geometry (RANSAC, RDP, skeleton analysis), and AI reasoning (SLM with fallback to cloud APIs). Each stage operates on validated Pydantic schema contracts, enabling independent development, testing, and replacement.

2. **Geometric calibration for chart value extraction**: RANSAC-based axis calibration with pixel-to-value mapping achieves deterministic, traceable value extraction. This fundamentally differs from end-to-end models that estimate values from pixel patterns.

3. **Large-scale academic chart dataset**: 32,364 classified charts from ∼4,000 arXiv papers, with structured geometric features (axis calibration, OCR with spatial roles, element detection). The derived SLM training dataset contains 268,799 ChatML conversation samples across 8 chart types.

4. **Language-extensible architecture for Vietnamese chart Q&A**: The pipeline is designed with a Core-first, Localize-second strategy where 12 out of 17 components are fully language-agnostic. Vietnamese support is achieved through configuration changes (OCR language), additive extensions (keyword dictionaries, prompt templates), and targeted data generation (Vietnamese QA samples) – without modifying any core pipeline logic.

5. **Comprehensive evaluation**: Chart detection at 93.5% mAP@0.5, classification at 94.14% accuracy, and 100% batch extraction success rate across the full 32,364-chart corpus with 92.6% overall pipeline confidence.

7.2 Achievement of Objectives

Table 21: Achievement of project objectives.

Objective	**Target**	**Achieved**	**Status**
Detection mAP@0.5	$> 85\%$	93.5%	Exceeded
Classification accuracy	$> 90\%$	94.14%	Exceeded
Extraction confidence	$> 90\%$	92.6%	Exceeded
Training dataset size	$> 200{,}000$	268,799	Exceeded
Consumer GPU deployment	RTX 3060	Confirmed	Met
Vietnamese Q&A support	End-to-end	Architecture ready	In progress
SLM fine-tuning	Complete	In progress	Pending
Public benchmark eval	ChartQA/PlotQA	Not yet	Pending

Five of seven objectives were exceeded. The Vietnamese Q&A architecture is in place (12/17 components language-agnostic, localization layers designed), pending QA data generation and SLM fine-tuning. The remaining items are in progress.

7.3 Future Work

1. **Vietnamese chart Q&A deployment**: Execute the full Vietnamese localization plan: configure OCR for Vietnamese diacritical marks, generate 5,000–10,000 Vietnamese QA samples via Data Factory, fine-tune SLM with combined English + Vietnamese data, and evaluate Vietnamese chart understanding accuracy end-to-end.

2. **SLM fine-tuning and evaluation**: Complete QLoRA training of Qwen-2.5-1.5B on the 268,799-sample dataset (English) plus Vietnamese supplement. Target: JSON valid rate >95%, field accuracy >90%, Vietnamese output fluency >90%, latency <2s per chart.

3. **Public benchmark comparison**: Evaluate on ChartQA [13] and PlotQA [14] for direct comparison with DePlot [1] and MatCha [2].

4. **Ablation study**: Systematically measure the contribution of the negative image transform, adaptive RDP vectorization, RANSAC calibration, and SLM reasoning through controlled experiments.

5. **OCR optimization**: Implement batch processing and selective re-execution to reduce Stage 3 latency from ∼5.0s to <2.5s.

6. **Vietnamese domain expansion**: Collect chart images from Vietnamese government reports, business dashboards, and academic publications to build a Vietnamese chart benchmark dataset.

7. **Production deployment**: Integrate the Celery task queue and FastAPI server for asynchronous pipeline execution with PostgreSQL state management.

8. **Dataset release**: Prepare the training dataset (English + Vietnamese) and model weights for public release to support community research.

References

[1] F. Liu, F. Piccinno, S. Krichene, C. Pang, K. Lee, M. Joshi, Y. Choi, J. M. Eisenschlos, DePlot: One-shot visual language reasoning by plot-to-table translation, in: Findings of the Association for Computational Linguistics: ACL 2023, 2023, pp. 10381–10399.

[2] F. Lee, F. Piccinno, S. Krichene, J. M. Eisenschlos, MatCha: Enhancing visual language pretraining with math reasoning and chart derendering, in: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics, 2023.

[3] M. Savva, N. Kong, A. Chhajta, L. Fei-Fei, M. Agrawala, J. Heer, ReVision: Automated classification, analysis and redesign of chart images, in: Proceedings of the 24th Annual ACM Symposium on User Interface Software and Technology, 2011, pp. 393–402.

[4] G. Jocher, A. Chaurasia, J. Qiu, Ultralytics YOLOv8, GitHub repository (2023). `https://github.com/ultralytics/ultralytics`.

[5] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 770–778.

[6] M. A. Fischler, R. C. Bolles, Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography, Communications of the ACM 24 (1981) 381–395.

[7] D. H. Douglas, T. K. Peucker, Algorithms for the reduction of the number of points required to represent a digitized line or its caricature, Cartographica: The International Journal for Geographic Information and Geovisualization 10 (1973) 112–122.

[8] U. Ramer, An iterative procedure for the polygonal approximation of plane curves, Computer Graphics and Image Processing 1 (1972) 244–256.

[9] T.-C. Lee, R. L. Kashyap, C.-N. Chu, Building skeleton models via 3-d medial surface/axis thinning algorithms, CVGIP: Graphical Models and Image Processing 56 (1994) 462–478.

[10] G. Bradski, The OpenCV library, in: Dr. Dobb's Journal of Software Tools, 2000.

[11] A. Yang, B. Yang, et al., Qwen2.5: A party of foundation models, arXiv preprint arXiv:2412.15115 (2024).

[12] T. Dettmers, A. Pagnoni, A. Holtzman, L. Zettlemoyer, QLoRA: Efficient finetuning of quantized language models, Advances in Neural Information Processing Systems 36 (2024).

[13] A. Masry, D. X. Long, J. Q. Tan, S. Joty, E. Hoque, ChartQA: A benchmark for question answering about charts with visual and logical reasoning, in: Findings of the Association for Computational Linguistics: ACL 2022, 2022, pp. 2263–2279.

[14] N. Methani, P. Ganguly, M. M. Khapra, P. Kumar, PlotQA: Reasoning over scientific plots, Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (2020) 1527–1536.

[15] Z.-Q. Chen, Q. Shen, W. Ma, ChartReader: A unified framework for chart derendering, arXiv preprint arXiv:2210.09330 (2022).

[16] J. Redmon, S. Divvala, R. Girshick, A. Farhadi, You only look once: Unified, real-time object detection, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 779–788.

[17] Y. Du, C. Chen, R. Li, et al., PP-OCR: A practical ultra lightweight ocr system, in: arXiv preprint arXiv:2009.09941, 2020.

[18] JaidedAI, EasyOCR: Ready-to-use ocr with 80+ supported languages, GitHub repository (2020). `https://github.com/JaidedAI/EasyOCR`.

[19] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, LoRA: Low-rank adaptation of large language models, International Conference on Learning Representations (2022).

[20] M. Tan, Q. V. Le, EfficientNet: Rethinking model scaling for convolutional neural networks, in: Proceedings of the 36th International Conference on Machine Learning, 2019, pp. 6105–6114.